

# Scheduling with Two Non-Unit Job Lengths Is NP-Complete, Formalized

Yuval Itzhaki

Claude

## Abstract

This work formalizes a theorem of Elffers and de Weerd [1]: non-preemptive scheduling of jobs with release times and deadlines on a single machine,  $1 \mid r_j \mid L_{\max}$ , is polynomial-time solvable when all jobs have the same length, and when the jobs have two lengths of which the shorter is 1 [3]. Elffers and de Weerd settle the remaining case: for every fixed pair of integer job lengths  $p > q > 1$ , the problem restricted to the job lengths  $\{p, q\}$  is NP-complete. The reduction produces only numbers polynomial in the size of the input, which is why they call the problem strongly NP-complete.

## 1 The Problem

A single machine processes  $n$  jobs, one at a time and without preemption. Job  $j$  carries a release time  $r_j$ , a deadline  $d_j$ , and a processing time  $p_j$ , all non-negative integers.

**Definition 1** (Scheduling with Release Times and Deadlines). An *instance* is a finite set of jobs  $([r_j, d_j], p_j)$ . A *schedule* is a start-time assignment  $t$ ; it is *feasible* if every job runs inside its availability window,

$$r_j \leq t(j) \leq d_j - p_j \quad \text{for every job } j,$$

and the jobs' execution intervals  $[t(j), t(j) + p_j]$  are pairwise disjoint. An instance is *schedulable* if it admits a feasible schedule.

Figure 1 shows a feasible schedule of a two-job instance.

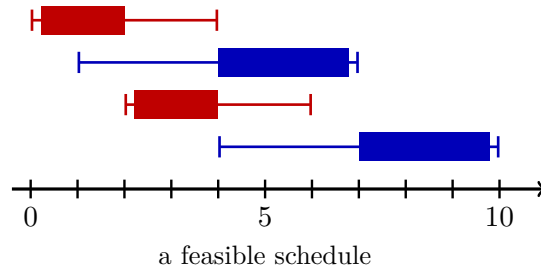


Figure 1: Two jobs, their availability windows, and one feasible placement.

A feasible schedule exists exactly when  $L_{\max} \leq 0$  for the usual maximum-lateness objective, which in turn holds exactly when every job can be made on time,  $\sum_j U_j = 0$ ; this is the sense in which the decision problem above is  $1 \mid r_j \mid L_{\max}$ .

Elffers and de Weerd organize the complexity landscape of this problem around the *set of lengths that occur*,  $P(S) = \{p_j : j \in S\}$ : the number of distinct lengths,  $\#P := |P(S)|$ , and the longest length,  $p_{\max} := \max P(S)$ . The general problem is strongly NP-hard by a reduction from 3-PARTITION, so the interesting question is how small  $P(S)$  can be made before the problem turns easy. A single length ( $\#P = 1$ ) is trivially polynomial; two lengths of which the shorter is 1

is also polynomial, by an LP-rounding algorithm of Sgall [3]. Elffers and de Weerd’s remaining case, two lengths neither of them 1, is the subject of this formalization.

Sgall’s survey of open problems in throughput scheduling poses exactly this question: is there a polynomial-time algorithm for  $1 \mid r_j, p_j \leq c \mid \sum U_j$  for constant  $c$ , i.e. does  $P(S)$  staying small enough keep the problem easy [3]? Mnich and van Bevern’s survey of parameterized machine scheduling restates it as Open Problem 3: are  $1 \mid r_j \mid \sum U_j$  and  $1 \mid r_j \mid \sum w_j U_j$  W[1]-hard parameterized by  $p_{\max}$ , membership in even XP being open [5]. Taking  $p = 2q$  in Theorem 1, for every  $q \geq 2$ , contradicts Vakhania’s claimed pseudo-polynomial algorithm for  $p_j \in \{q, 2q\}$  [4].

## 2 The Main Theorem

Instances are read as finite binary words: a natural number is its digits preceded by their count in unary, and a scheduling instance is the concatenation of its jobs’ numbers, each number’s sign bit first.

**Definition 2** (Instances as Words). *Scheduling on the lengths  $\{p, q\}$ , for fixed  $p, q$ , is the language of binary words encoding an instance all of whose processing times lie in  $\{p, q\}$  and which is schedulable.*

**Theorem 1** (Elffers–de Weerd). *Let  $p > q > 1$  be two integer job lengths. Scheduling on the lengths  $\{p, q\}$  is NP-complete.*

The membership half of Theorem 1 holds unconditionally, for every pair of lengths:

**Theorem 2.** *Scheduling on the lengths  $\{p, q\}$  belongs to NP, for every  $p, q$ .*

A certificate is a schedule with integer start times; feasibility confines every start time between a release time and a deadline of the instance, so the certificate is no longer than the instance itself, and checking it is immediate. Hardness is the substance of the theorem, and the rest of this paper is its proof.

**Observation 1.** *Since a feasible schedule exists exactly when  $\sum_j U_j = 0$  (Definition 1), Theorem 1 says that  $1 \mid r_j, p_j \leq p \mid \sum U_j$  is NP-hard already for a constant  $p$ , i.e. para-NP-hard parameterized by  $p_{\max}$ . In particular there is no polynomial-time algorithm for any fixed bound on  $p_{\max}$  unless  $P = NP$ , which settles Sgall’s question and Mnich and van Bevern’s Open Problem 3 [3, 5].*

## 3 The Shape of the Proof

The reduction from satisfiability does not go directly to scheduling. Deciding a clause needs a disjunction between two jobs whose deadlines can be far apart, and plain release times and deadlines cannot say that. Elffers and de Weerd introduce an intermediate problem that can.

**Definition 3** (The Auxiliary Problem  $AUX(p, q)$ ). An instance of  $AUX(p, q)$  consists of ordinary jobs as in Definition 1, together with  $N$  pending pairs  $(J_{p,i}, J_{q,i})_{i=1}^N$ . Both jobs of a pair are released at 0;  $J_{p,i}$  has length  $p$  and  $J_{q,i}$  has length  $q$ . Each pending job carries two deadlines, an *early* one  $d'$  and a *late* one  $d$ , with  $d' \leq d$ . The instance is *ordered* when, for every  $i$ ,

$$d'_{p,i} \leq d_{p,i} \leq d'_{p,i+1}, \quad d'_{q,i} \leq d_{q,i} \leq d'_{q,i+1}, \quad d_{p,i} \leq d'_{q,i},$$

and it has a *solution* if there is a feasible schedule with respect to the late deadlines in which, for every  $i$ , at least one of  $J_{p,i}, J_{q,i}$  meets its early deadline.

Figure 3 shows one connected pending pair.

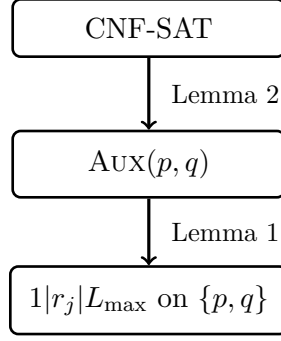


Figure 2: SAT reduces to the auxiliary problem  $AUX(p, q)$  (Lemma 2), which reduces to scheduling on  $\{p, q\}$  (Lemma 1).

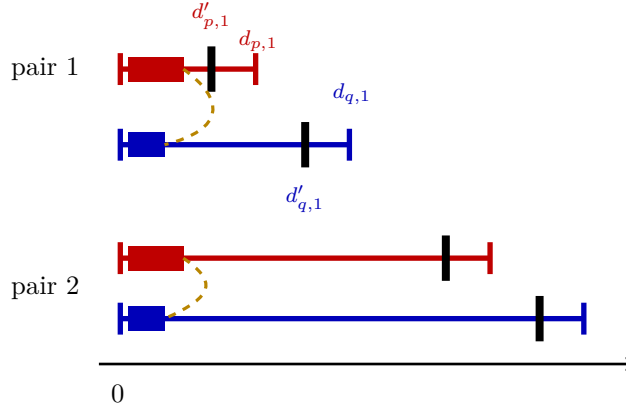


Figure 3: A connected pending pair: at least one of the two jobs must meet its early deadline.

## 4 From Satisfiability to $Aux(p, q)$

**Lemma 2.** For job lengths  $p > q > 1$ ,  $AUX(p, q)$  is NP-complete; the reduction from every language in NP factors through a polynomial-time many-one reduction from satisfiability.

*Construction.* Fix a CNF formula  $F$  with  $n$  variables and  $m$  clauses  $C_0, \dots, C_{m-1}$ . The time line is cut into  $2n$  sections of equal length  $S = (p + 2q) + m(p + q) + 1 + q$ , one for every literal, in the order  $x_0, \neg x_0, x_1, \neg x_1, \dots$ . A section consists of a *literal block* of total job length  $p + 2q$ , then one *clause block* of total job length  $p + q$  for every clause, then one unit of idle time, then a *separator*: an ordinary short job pinned to the last  $q$  time units of the section. Within a literal block, the jobs are arranged so that a feasible schedule has exactly two shapes, differing by one unit of idle time: one shape leaves no idle time in the block and corresponds to the literal being TRUE, the other leaves exactly one unit of idle time and corresponds to FALSE. A short ordinary job of length  $q \geq 2$  cannot absorb that one unit of slack on its own, so the idle time, once created, propagates rightwards through every clause block that follows in the section: a clause block lets the slack through unless one of its literals is set to make the clause true, in which case the block's pending pair is tethered so as to absorb it. A clause is satisfied exactly when some literal's block feeds its clause block enough slack to stop the propagation before it reaches the separator, which is why the separator's own feasibility encodes satisfiability of the whole formula.

Figure 4 shows all four block shapes: the selection gadgets  $V^+, V^-$  for a positive and a negative literal, and the clause gadgets  $C_{\text{act}}, C_{\text{inact}}$  for a clause that does, respectively does not, contain that literal.

Complementary literals  $x_i, \neg x_i$  are kept consistent by a connected pending pair tethering

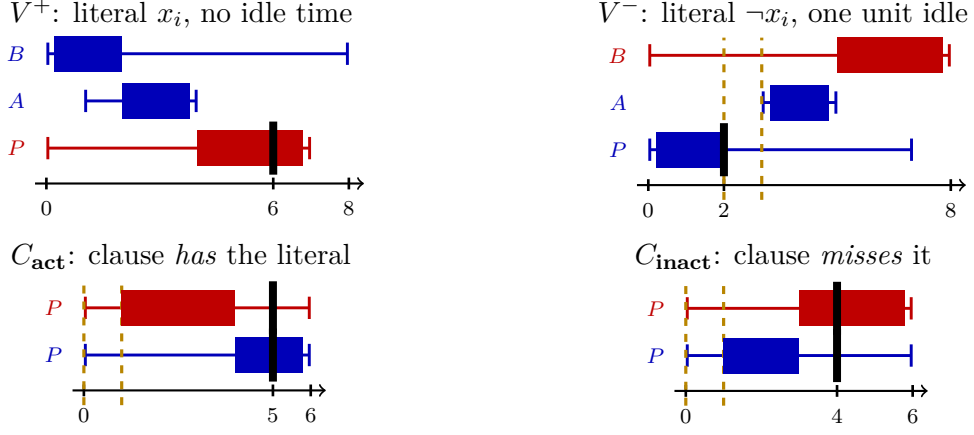


Figure 4: The four block shapes.  $B$ ,  $A$ ,  $P$  are ordinary jobs; the two rows marked  $P$  in a clause gadget are the pending jobs of its connected pair, as in Figure 3.

their two literal blocks, so that one is forced TRUE exactly when the other is forced FALSE. Figure 5 assembles four full sections,  $x_1, \neg x_1, x_2, \neg x_2$ , for the formula  $(x_1 \vee x_2) \wedge (\neg x_1 \vee x_2)$ , every tethered pair shown.

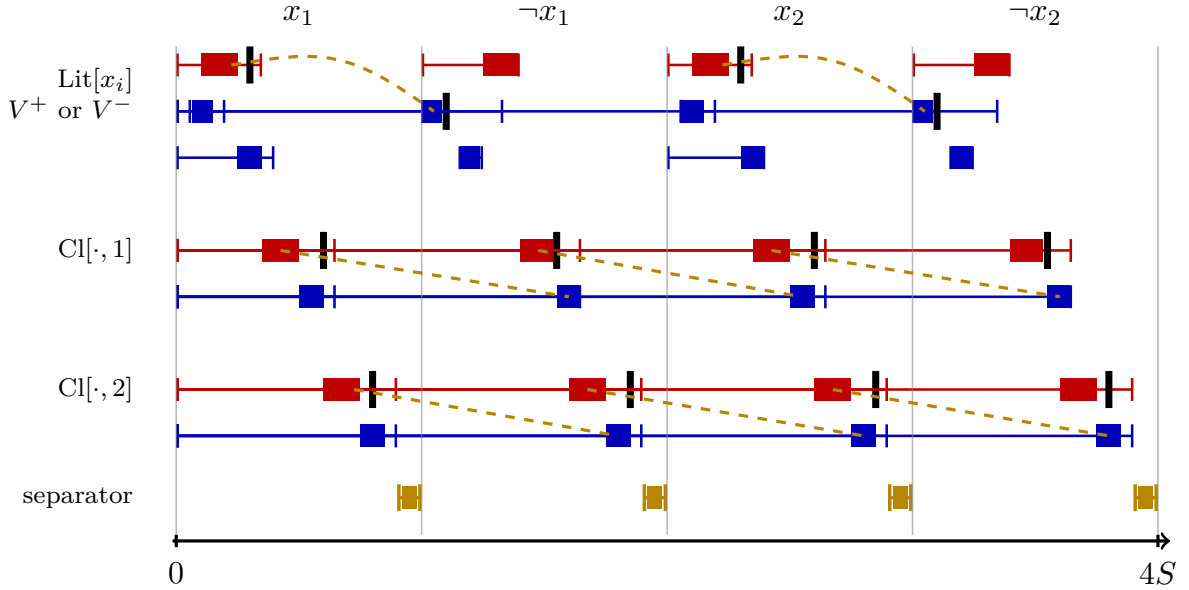


Figure 5: Four sections of the construction for  $p = 3$ ,  $q = 2$ : two variables, each contributing a satisfied and an unsatisfied clause slot to the two clauses of  $(x_1 \vee x_2) \wedge (\neg x_1 \vee x_2)$ .

Correctness is proved as two separate statements:

- a word belongs to SAT if and only if its image under the reduction belongs to  $\text{AUX}(p, q)$ ;
- the constructed instance is ordered, so that Definition 3's hypothesis is met;

the forward and converse directions of the first statement are themselves separate proofs, satisfiable  $\Rightarrow$  solvable and solvable  $\Rightarrow$  satisfiable.  $\square$

**Proposition 1** (The Numbers Stay Small). *Every release time, deadline, and early or late deadline of the constructed instance is at most  $2nS$ , the length of the time line, a polynomial in the size of the formula.*

Membership of  $\text{AUX}(p, q)$  in NP, and the polynomial running time of the reduction, are proved by exhibiting word-RAM programs: a verifier for  $\text{AUX}(p, q)$  running in polynomial time, and the reduction itself running in polynomial time. Composing these with Cook–Levin gives NP-hardness of  $\text{AUX}(p, q)$ , and, with membership, its NP-completeness.

## 5 From $\text{Aux}(p, q)$ to Scheduling

**Lemma 1.** *For job lengths  $p > q \geq 1$ ,  $\text{AUX}(p, q)$  is polynomial-time reducible to scheduling on the lengths  $\{p, q\}$ .*

*Construction.* Fix  $w = q$  and let  $t_i = -(p + q + w)i$  for  $i = 1, \dots, N$ . For every pending pair  $i$  the constructed instance contains five jobs: a *separator* with availability interval  $[t_i + p + q, t_i + p + q + w]$  and length  $w$ , which is thereby pinned in place; an *inner* long job  $([t_i + q, d'_{p,i}], p)$  and an *outer* long job  $([t_i, d_{p,i}], p)$ ; an *inner* short job  $([t_i + p, d'_{q,i}], q)$  and an *outer* short job  $([t_i, d_{q,i}], q)$ . The ordinary jobs are kept unchanged. The separators cut the time before 0 into  $N$  bins, each with room for exactly one long and one short job; the four jobs of a pair are nested, with the inner ones carrying the early deadlines. A feasible schedule parks two jobs of each pair in its bin and runs the other two, one long and one short, after time 0. The inner long job and the inner short job of a pair do not both fit into the bin together, so one of the two jobs running after 0 is an inner job and meets an early deadline: exactly Definition 3’s condition on connected pairs.

Figure 6 shows bin 1 with one proper pairing parked inside it; the complementary pairing then runs after time 0 and meets its early deadline.

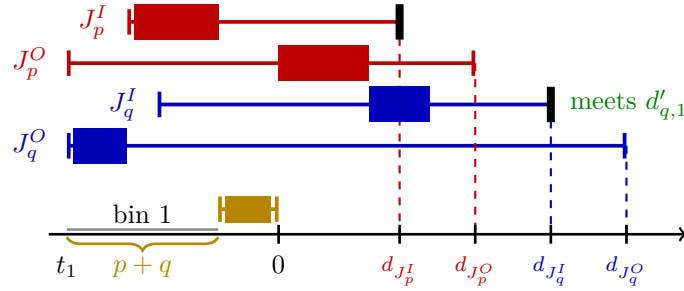


Figure 6: Bin 1, pinned by its separator (amber), holding the inner short job  $J_q^I$  and the inner long job  $J_p^I$ ; the outer long job  $J_p^O$  and outer short job  $J_q^O$  run after time 0, and  $J_q^I$  meets its early deadline  $d'_{q,1}$ .

**Proposition 2** (The Construction Is Correct). *For job lengths  $p > q \geq 1$ , an ordered instance of  $\text{AUX}(p, q)$  has a solution exactly when the stacked instance has a feasible schedule.*

**Proposition 3.** *The stacked instance uses only the job lengths  $p$  and  $q$ .*

**Proposition 4** (The Numbers Stay Small). *If every release time and deadline of an ordered instance  $A$  is at most  $T$ , then every release time and deadline of the stacked instance is within  $T + (p + 2q)N$  of 0, where  $N$  is the number of pending pairs.*

□

Together with Proposition 1’s bound on  $\text{AUX}(p, q)$ ’s own numbers, this bounds every number produced by composing the two reductions by a polynomial in the size of the original formula, since  $p$  and  $q$  are fixed constants, the ingredient Elffers and de Weerd’s source uses to call the hardness *strong*.

As for Lemma 2, three separate proofs give, respectively, the reduction’s correctness, its polynomial running time, and the many-one reduction Lemma 1 states.

## 6 Integer Start Times

**Proposition 5.** *A scheduling instance is schedulable by a real-valued start-time assignment exactly when it is schedulable by an integer one.*

*Proof.* Rounding every start time up to the next integer only shortens the gap to each job’s deadline, and keeps execution intervals disjoint because every release time and deadline is already an integer. This settles a remark Elffers and de Weerdts make in passing: nothing is lost by restricting Definition 1 to integer schedules, as the rest of this paper does.  $\square$

## 7 Putting It Together

Composing Lemma 2 and Lemma 1 gives Theorem 1’s NP-hardness:

- every language in NP many-one reduces to scheduling on  $\{p, q\}$  through  $\text{AUX}(p, q)$ ;

and, with membership, the theorem itself:

- scheduling on the lengths  $\{p, q\}$  is NP-complete.

## 8 Formalization Notes

Hardness rests on the archive’s own proof of the Cook–Levin theorem, not on a literature axiom: every language in NP many-one reduces to satisfiability as a cited result of THE COOK–LEVIN THEOREM, composed here with the two reductions above. The polynomial-time bounds of Lemma 2 and Lemma 1 are proved by exhibiting explicit programs on THE WORD RAM and transferring their running time to Turing-machine polynomial time via the proved equivalence of the two models; composing the resulting machines over possibly different alphabets uses a polynomial-time Turing-machine composition proved in CLASSICAL COMPLEXITY CLASSES.

## 9 Discussion

Single-machine scheduling with release times and deadlines is polynomial-time solvable when every job has the same length, and remains so when the lengths are drawn from  $\{1, p\}$  for any  $p$  [3]. Theorem 1 shows that this tractability is fragile: once both lengths exceed 1, the problem is NP-complete, for every fixed pair  $p > q > 1$ . Since  $p$  and  $q$  are constants rather than part of the input, this rules out a polynomial-time algorithm at any fixed pair of lengths, settling Sgall’s open question and Mnich and van Bevern’s Open Problem 3 (Observation 1). The dividing line between tractable and NP-complete sits exactly at excluding the unit length.

## References

- [1] J. Elffers and M. de Weerdts. Scheduling with two non-unit job lengths is NP-complete. arXiv:1412.3095, 2017. <https://doi.org/10.48550/arXiv.1412.3095>.
- [2] S. A. Cook. The complexity of theorem-proving procedures. In *Proceedings of the Third Annual ACM Symposium on Theory of Computing*, 151–158, 1971. <https://doi.org/10.1145/800157.805047>.
- [3] J. Sgall. Open problems in throughput scheduling. In *Algorithms, ESA 2012*, 2–11, 2012. [https://doi.org/10.1007/978-3-642-33090-2\\_2](https://doi.org/10.1007/978-3-642-33090-2_2).

- [4] N. Vakhania. Single-machine scheduling with release times and tails. *Annals of Operations Research* 129:253–271, 2004. <https://doi.org/10.1023/B:ANOR.0000030692.69147.e2>.
- [5] M. Mnich and R. van Bevern. Parameterized complexity of machine scheduling: 15 open problems. *Computers & Operations Research* 100:254–261, 2018. <https://doi.org/10.1016/j.cor.2018.07.020>.